

Let's Embed Your Knowledge into AI by Trial and Error Instead of Annotation

Osamu Saisho
osamu.saisho.vm@hco.ntt.co.jp
NTT
Tokyo, Japan

Keiichiro Kashiwagi
keiichirou.kashiwagi.yk@hco.ntt.co.jp
NTT
Tokyo, Japan

Koki Mitani
koki.mitani.xg@hco.ntt.co.jp
NTT
Tokyo, Japan

ABSTRACT

The large annotation cost has always remained a challenge for machine learning applications in ubiquitous computing. Programmatic weak supervision (PWS) is a promising solution, but an efficient method and system for domain experts to build it interactively from scratch has not yet been developed. This paper shows the necessity of not only increasing PWS sources but also modifying or removing them and the method to achieve this. Also, we developed a prototype system to build artificial intelligence through trial and error with PWS. The user study with 15 users verified the necessity and effects of modifying and removing PWS sources through this method.

CCS CONCEPTS

• **Human-centered computing** → *Ubiquitous and mobile computing systems and tools.*

KEYWORDS

programmatic weak supervision, human-in-the-loop machine learning, bidirectional active learning

ACM Reference Format:

Osamu Saisho, Keiichiro Kashiwagi, and Koki Mitani. 2022. Let's Embed Your Knowledge into AI by Trial and Error Instead of Annotation. In *Proceedings of the 2022 ACM International Joint Conference on Pervasive and Ubiquitous Computing (UbiComp/ISWC '22 Adjunct)*, September 11–15, 2022, Cambridge, United Kingdom. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3544793.3560352>

1 INTRODUCTION AND RELATED WORK

The large annotation cost has remained a challenge for constructing new machine learning applications in ubiquitous computing. Programmatic weak supervision (PWS) is a promising method to reduce the cost because it removes the necessity of labeling individual data [8, 14]. PWS is a user-defined noisy source program such as heuristics and knowledge bases that each vote labels for data subset. The representative of PWS source is a labeling function (LF), and Snorkel [7] is often used for implementation. Since the output of LFs is noisy and contradictory, the learning model consists of two components: a label model, which aggregates the noisy labels to obtain training labels, and an end model, which uses

these labels for learning and classification. A joint model has also been developed that simultaneously performs label aggregation and final classification [9].

To maximize the effectiveness of PWS, domain experts should be supported to efficiently implement their knowledge as PWS sources from scratch because the work cost of creating an LF itself is heavier than that of attaching a label. However, most studies on PWS implicitly give well-designed PWS sources and do not consider the actual cost to create them. Only a few iterative methods have shown the feasibility of combining active learning and PWS creation [11, 13]. In each active learning loop, a domain expert implements a new LF by referring to the suggested data samples with large labeling utility. Though it is a simple and user-friendly method, there is still an issue of failing to handle the unexpected noisy and harmful LFs. It is essential to modify LFs posteriorly when building new applications from scratch. In fact, some widely used datasets constructed by conventional labeling contain a few percent of label noise [6]. Even more so with a handmade PWS, noiseless sources cannot be created in a single shot.

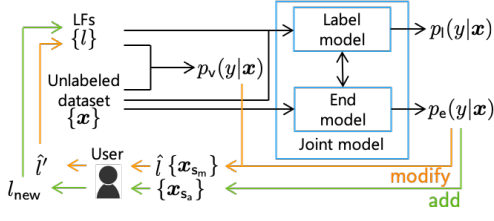
In terms of annotation system, interactive systems such as Label Studio [12] have been developed for conventional labeling. It can easily handle various data types and connect to machine learning models and databases. For PWS, some techniques have been proposed to avoid requiring programming skill of domain experts [2–4]. However, there are no support tools that provide the ability to think of hand-made PWS resources instead of conventional redundant annotation.

The main contributions of this paper are below. We propose a method to realize trial-and-error PWS construction by extracting a PWS source to be modified and verify its effectiveness using actual data from user studies with 15 users. We also build a prototype system for trial-and-error PWS construction with this method and active learning for PWS.

2 PROPOSED METHOD AND PROTOTYPE SYSTEM

Figure 1 shows the overview of the learning framework for trial-and-error construction of PWS. Note that the learning model and a loop for adding a new LF are derived from the active learning approach applied to the joint model [10]. In a loop for modifying a past LF, domain experts should be given the most harmful LF and the data samples that help the modification procedure. Suppose that unlabeled data $x \in \mathcal{X}$ and implemented LFs $l \in \mathcal{L}$ are given and the target classes are $y \in \mathcal{Y}$. Note that the case where l abstains from voting is expressed as $l(x) = -1$. This learning model estimates three output probability distributions after the model training in each loop: simple probability based on the share of LF

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
UbiComp/ISWC '22 Adjunct, September 11–15, 2022, Cambridge, United Kingdom
© 2022 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-9423-9/22/09.
<https://doi.org/10.1145/3544793.3560352>


Figure 1: Overview of loops in the learning framework

Question classification for TREC dataset		
class info	Sampled data	Result
0: location	How do you make a paintball?	
1: human	How was the element calcium named?	
2: numeric	How can you raise your IQ?	
3: entity	How successful is aromatherapy?	
4: abbreviation	How does a rainbow form?	
5: description		
LF pool		Backend status
If_shame If_who If_short If_stand If_num		Training finished
Labeling Function		
1: (labeling_function())		
Verify Submit Reload		

Figure 2: Screen for adding a new LF

Question classification for TREC dataset		
class info	Sampled data	Result
0: location	How many people in the world speak French?	5
1: human	How many colleges are in Norway?	5
2: numeric	How much was the minimum wage in 1993?	5
3: entity	How many years ago did the ship Titanic sink?	5
4: abbreviation	How much snow equals an inch of rain?	5
5: description		
LF pool		Backend status
If_shame If_who If_short If_stand If_num If_how		Training finished
Labeling Function		
1: (labeling_function()) 2: def If_how(x): 3: if "how" in x.lower(): 4: return 5 5: else: 6: return -1		
Verify Resubmit Remove Reload		

Figure 3: Screen for modifying a past LF

votes $p_v(y|x)$, the output probability of label model $p_l(y|x)$, and that of end model $p_e(y|x)$. On the basis of these observations, the distance between $p_v(y|x)$ and $p_e(y|x)$ can be considered to represent the dissimilarity between the votes of LFs and end model classification results at x . Then the indicator of LF harmfulness, $h(l)$, is defined as $h(l) = \sum_{x \in \{x | l(x) \neq -1\}} D_{KL}(p_v(y|x) || p_e(y|x))$, i.e., the sum of KL divergence between $p_v(y|x)$ and $p_e(y|x)$ with respect to x for which l votes any label. Consequently, the LF estimated to be most harmful, $\hat{l} = \operatorname{argmax}_{l \in \mathcal{L}} h(l)$, and a few data samples with the largest KL divergence $\{x_{s_m}\} \subset \{x | \hat{l}(x) \neq -1\}$ are suggested to a domain expert. The incorporation of this modification loop into the framework enables efficient and trial-and-error user interaction in PWS construction.

Our prototype system enables users to efficiently construct PWS through trial and error by iterative addition and modification of LFs as shown in Figures 2 and 3. This system has a learning framework described above in backend and is connected to it via Label Studio API [12]. In a loop for adding a new LF, when “Reload” is pressed, the five data extracted by the active learning sampler will be displayed in “Sampled data”. Users then implement a new LF in “Labeling Function” while referring to the samples and check if the LF works as intended by pressing “Verify”, which shows the votes of the LF in “Result”. The LF will be sent to the LF pool in the backend by pressing “Submit”, and model re-training will be executed automatically. In a loop for modifying a past LF, when “Reload” is pressed, the LF and the five data extracted by our method described above are displayed in “Labeling Function” and “Sampled data” with its votes in “Result”. The user can modify or remove the LF while referring to these information. Pressing “Resubmit” or “Remove” is a trigger to reflect the change to the LF pool in backend and to execute model re-training.

Figures 2 and 3 show the example loops in question classification for TREC dataset [5] for easy understanding. Note that it is not required to operate the addition and modification of the same LF continuously. With referring to the five samples starting with “how”, “If_how” was added to the LF pool in Figure 2, but as shown in Figure 3, it is too noisy. As a result, “If_how” and other five samples starting with “how many” or “how much” are displayed for the user to modify “If_how” posteriorly so that it can correctly vote for these samples as well.

3 EXPERIMENT

To verify the necessity and the effectiveness of our method, we conducted user studies with 15 users on two tasks: SMS spam detection [1] and TREC question classification [5]. The users individually implemented 15 LFs for each task using the active learning for PWS framework. We conducted three verifications using these data. To prove the importance of the trial-and-error in PWS construction, we observed the ratio of label noise. Next, we verified the performance of our method to estimate the most harmful LF without any ground truth. Finally, we examined the efficacy of our method for the classification performance of the end model in two cases: simply remove the extracted LF (“remove once”) and completely eliminate label noise from the data the LF votes on (“correct once”), which is an ideal but unrealistic case.

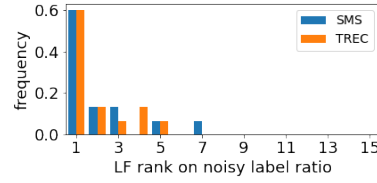
Table 1 shows the mean and worst noisy label ratio for each user in the form of mean $\pm$ standard deviation, respectively. Focusing on the worst LFs for each user, on average, they voted more for wrong labels than the correct label. Figure 4 shows the rank frequency of LFs that our method extracted among the 15 LFs created by each user in terms of the label noise ratio. The results demonstrate that our method achieves 60% accuracy in the top-1 and more than 80% accuracy in the top-3 in both tasks. Table 2 shows the classification inaccuracy with and without our method. For comparison, “not modify” is the result before applying our method. The results revealed that even only removing the extracted LF improved performance, even though the number of LFs was reduced, i.e., from 15 to 14. Also, a larger improvement can be expected by modifying it correctly. All of these results indicate the necessity of a posterior modification for PWS, the effectiveness of our method for PWS modification, and its contribution to improve classification performance.

4 CONCLUSION

We proposed and evaluated an interactive method to construct a programmatic weak supervision and developed its prototype system. Our method enables efficient modification of PWS source by focusing on the gaps between output probability distributions of a learning model and realizes a trial-and-error procedure. The experimental results verify its necessity, advanced accuracy, and effectiveness for classification. For future work, we will realize dynamic optimal selection to add or modify a source and eliminate the necessity of programming skill. It is important to create a workable system along with a new learning framework especially for human-AI cooperations, which will generate synergies.

Table 1: Label noise ratio of each LF

	SMS	TREC
mean	0.09 ± 0.10	0.14 ± 0.09
worst	0.52 ± 0.41	0.67 ± 0.33

**Figure 4: Detective results of harmful LFs****Table 2: Inaccuracy of classification results**

	SMS	TREC
not modify	0.11 ± 0.07	0.44 ± 0.03
remove once	0.09 ± 0.03	0.40 ± 0.01
correct once	0.08 ± 0.01	0.36 ± 0.01

REFERENCES

- [1] Tiago Almeida, José Hidalgo, and Akebo Yamakami. 2011. Contributions to the Study of SMS Spam Filtering: New Collection and Results. In *Proc. of DocEng*. 259–262.
- [2] Eran Bringer, Abraham Israeli, Yoav Shoham, Alex Ratner, and Christopher Ré. 2019. Osprey: Weak Supervision of Imbalanced Extraction Problems without Code. In *Proceedings of the 3rd International Workshop on Data Management for End-to-End Machine Learning* (Amsterdam, Netherlands) (*DEEM'19*). Article 4, 11 pages. <https://doi.org/10.1145/3329486.3329492>
- [3] Sainyam Galhotra, Behzad Golshan, and Wang-Chiew Tan. 2021. Adaptive Rule Discovery for Labeling Text Data (*SIGMOD '21*). <https://doi.org/10.1145/3448016.3457334>
- [4] B. Hancock, P. Varma, S. Wang, M. Bringmann, P. Liang, and C. Ré. 2018. Training Classifiers with Natural Language Explanations. In *Proc. of ACL*. 1884–1895.
- [5] Xin Li and Dan Roth. 2002. Learning Question Classifiers. In *Proc. of COLING*. 1–7.
- [6] Curtis Northcutt, Anish Athalye, and Jonas Mueller. 2021. Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks. In *Proc. of NeurIPS*.
- [7] Alexander Ratner, Stephen Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. 2017. Snorkel: Rapid Training Data Creation with Weak Supervision. *Proc. VLDB Endow.* 11, 3 (2017), 269–282.
- [8] Alexander Ratner, Christopher Sa, Sen Wu, Daniel Selsam, and Christopher Ré. 2016. Data Programming: Creating Large Training Sets, Quickly. In *Proc. of NeurIPS*. 3567–3575.
- [9] Wendi Ren, Yinghao Li, Hanting Su, David Kartchner, Cassie Mitchell, and Chao Zhang. 2020. Denoising Multi-Source Weak Supervision for Neural Text Classification. In *Proc. of EMNLP*. 3739–3754.
- [10] Osamu Saisho. 2022. Probable Human Knowledge Elicitation for Efficient Human-in-the-Loop Text Classification without Any Label-Related Data (*In Proc. of PACLIC*).
- [11] Osamu Saisho, Takeshi Ohguro, Jingyu Sun, Hiroshi Imamura, Susumu Takeuchi, and Daigoro Yokozeki. 2021. Human Knowledge Based Efficient Interactive Data Annotation via Active Weakly Supervised Learning. In *Proc. of PerCom workshops*. 332–335.
- [12] Maxim Tkachenko, Mikhail Malyuk, Andrey Holmanyuk, and Nikolai Liubimov. 2020. Label Studio: Data labeling software. <https://github.com/heartexlabs/label-studio>
- [13] Benjamin Wang, Alex Ratner, Stephen Musmann, and Chris Ré. 2019. Interactive Programmatic Labeling for Weak Supervision. In *Proc. of KDD workshop*.
- [14] Jieyu Zhang, Cheng-Yu Hsieh, Yue Yu, Chao Zhang, and Alexander Ratner. 2022. A Survey on Programmatic Weak Supervision. *arXiv:2202.05433*